MySQL & SQL

IFT Toamasina

Niveau 2

2025

Introduction (01)

1. Quelques définitions

- Base de donnée

  Une base de données informatique est un ensemble de données qui ont été stockées sur un support informatique, et organisées et structurées de manière à pouvoir facilement consulter et modifier leur contenu.

- SGBD (Système de Gestion de Base de Données)

  Un Système de Gestion de Base de Données (SGBD) est un logiciel (ou un ensemble de logiciels) permettant de manipuler les données d'une base de données.

Introduction (02)

- Principale fonctionnalité d'un SGBD

  - ✓ Création et mises à jour de la structure de la base de données;
  - ✓ Administration de la base de données : gestion des utilisateurs, des droits d'accès etc;
  - ✓ Saisie et mises à jour des données;
  - ✓ Interrogation des données selon différents critères et/ou en effectuant des calculs.

- SGBDR (SGBD Relationnel)

  Le R de SGBDR signifie "relationnel". Un SGBDR est un SGBD qui implémente la théorie relationnelle. MySQL implémente la théorie relationnelle; c'est donc un SGBDR.

- SQL

  Le SQL (Structured Query Language) est un langage informatique qui permet d'interagir avec des bases de données relationnelles.

Interrogation de base de données en mode console (01)

2. Insertion de données

L'insertion de données dans une table s'effectue à l'aide de la commande INSERT INTO. Cette commande permet au choix d'inclure une seule ligne à la base existante ou plusieurs lignes d'un coup.

❑ Insertion d'une ligne dans une table

- Insérer une ligne en indiquant les informations pour chaque colonne existante (en respectant l'ordre).
  - Syntaxe

INSERT INTO nom_table VALUES ('valeur_1', 'valeur_2', …);

- Insérer une ligne en spécifiant les colonnes que vous souhaiter compléter.
  - Syntaxe

INSERT INTO nom_table (colonne_1, colonne_2,…)

VALUES('valeur_1', 'valeur_2', …);

Interrogation de base de données en mode console (02)

❑ Insertion de plusieurs lignes

Il est possible d'ajouter plusieurs ligne à un tableau avec une seule requête.

- Syntaxe

```
INSERT INTO nom_table (colonne_1, colonne_2,…)
VALUES ('valeur_1', 'valeur_2' ,…),
        ('valeur_1', 'valeur_2' ,…),
        ('valeur_1', 'valeur_2' ,…);
```

- Exemple

```
INSERT INTO parcours (nom)
VALUES ('Informatique'),
    ('Gestion');
```

Interrogation de base de données avec phpMyAdmin(07)

3. Interrogation de base de données

- Question sur les données;
- Moyen d'extraction des données de la base en fonction de plusieurs critères;
- La commande SELECT nous permet de sélectionner les données dans la base.

La syntaxe SQL simplifiée de l'instruction SELECT est la suivante :

SELECT [{ DISTINCT | DISTINCTROW } | ALL] *listeColonnes*
FROM *nomTable1* [, *nomTable2*] . . .
[WHERE *condition*]
[*clauseRegroupement*]
[HAVING *condition*]
[*clauseOrdonnancement*]
[LIMIT [*rangDépart*,] *nbLignes*] ;

Interrogation de base de données avec phpMyAdmin(08)

- DISTINCT | DISTINCTROW :jouent le même rôle, à savoir ne pas inclure les duplicatas.
- ALL prend en compte les duplicatas (option par défaut).
- listeColonnes : { * | *expression1* [[AS] *alias1*] [, *expression2* [[AS] *alias2*]...} :
  - ✓ * : extrait toutes les colonnes de la table.
  - ✓ *expression* : nom de colonne, fonction SQL, constante ou calcul.
  - ✓ *alias* : renomme l'expression (nom valable pendant la durée de la requête).
- FROM désigne la table (qui porte un alias ou non) à interroger.
- WHERE : restreint les lignes sélectionnées à celles pour lesquelles la condition est vraie.
- HAVING : restreint les groupes de lignes renvoyés à ceux pour lesquels la condition spécifiée est vraie.
- LIMIT pour limiter le nombre de lignes après résultat.

Interrogation de base de données avec phpMyAdmin(11)

Remarque :

Alias

Dans les requêtes de sélection, vous pouvez définir des alias pour les noms des bases, des tables ou des colonnes. Lors de l'affichage, les noms affichés sont ceux des alias et non ceux des colonnes de la table.

a. Restriction du nombre de ligne

- Pour appliquer une restriction, il suffit d'ajouter la clause WHERE à la commande SELECT.
- La clause WHERE ("où" en anglais) permet de restreindre les résultats selon des critères de recherche.
- Syntaxe :

  SELECT col1, col2, … FROM table WHERE condition

Interrogation de base de données avec phpMyAdmin(12)

❑ Les operateurs de comparaison

Opérateur	Description
=	Égalité des nombres ou des chaînes de caractères
<> Ou !=	Différent de
<=	Inferieur ou égal à
<	Inferieur
>=	Supérieur ou égal à
>	Supérieur
IS NULL	Retourne TRUE si la valeur est NULL.
IS NOT NULL	Retourne TRUE si la valeur n'est pas NULL.
express BETWEEN min AND max	Retourne TRUE si la valeur est comprise entre min et max (limites comprises).
express LIKE ' motif'	Retourne la valeur TRUE si l'expression est conforme au motif défini.
express IN(val1, val2, . . .)	Retourne TRUE si la valeur est incluse dans l'ensemble des valeurs listées.

Interrogation de base de données avec phpMyAdmin(13)

❑ Les operateurs logiques

Operateur	Description
NOT express ou ! express	Le NON logique renvoie FALSE si express vaut TRUE et réciproquement ou NULL si express vaut NULL.
express1 AND express2	Le ET logique renvoie TRUE si les deux opérandes sont différents de FALSE et de NULL et renvoie FALSE s'ils sont tous deux FALSE. Dans les autres cas, l'évaluation est NULL.
express1 OR express2	Le OU logique renvoie TRUE si au moins un des opérandes vaut TRUE, NULL si l'un des deux est NULL et FALSE dans les autres cas.
express1 XOR express	Le OU exclusif logique renvoie NULL si l'un des opérandes vaut NULL, TRUE si un seul des deux vaut TRUE et FALSE sinon.

Interrogation de base de données avec phpMyAdmin(14)

- **Illustration à l'aide des tables de vérité**:

Entrées		Sortie
A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Entrées		Sortie
A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

Entrée	Sortie
A	NOT A
0	1
1	0

Entrées		Sortie
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Interrogation de base de données avec phpMyAdmin(15)

❑ Les fonctions statistiques

Fonction	Description
AVG(colonne)	Retourne la moyenne des valeurs de la colonne précisée.
COUNT(colonne)	Retourne le nombre de lignes dont la valeur n'est pas NULL dans la colonne précisée. COUNT(*) retourne le nombre total de lignes, même si certaines ont la valeur NULL.
COUNT(DISTINCT colonne)	Retourne le nombre de lignes ayant une valeur non NULL et distincte (en éliminant les doublons).
MAX(colonne)	Retourne la valeur maximale de la colonne précisée.
MIN(colonne)	Retourne la valeur minimale de la colonne précisée.
SUM(colonne)	Retourne la somme des valeurs de la colonne précisée.

Interrogation de base de données avec phpMyAdmin(16)

❑ Le regroupement (GROUP BY)

La clause GROUP BY utilisée dans une commande SELECT permet de regrouper des lignes ayant une caractéristique commune. Nous obtenons de la sorte des sous-ensembles de lignes de la table auxquels nous pouvons appliquer des opérations telles que les fonctions statistiques.

- Syntaxe :

```
SELECT colonne1, fonction(colonne2)
        FROM table
        GROUP BY colonne1
```

- Exemple :

```
SELECT COUNT(`id_etudiant`) AS nb_etudiant
        FROM `etudiant`
        GROUP BY `ref_parcours`
```

Interrogation de base de données avec phpMyAdmin(17)

❑ Le regroupement et commande HAVING

La condition HAVING en SQL est presque similaire à WHERE à la seule différence que HAVING permet de filtrer en utilisant des fonctions telles que SUM(), COUNT(), AVG(), MIN() ou MAX().

- Syntaxe:

```
SELECT colonne1, fonction(colonne2)
        FROM nom_table
        GROUP BY colonne1
        HAVING fonction(colonne2) operateur valeur
```

- Exemple :

```
SELECT COUNT(`id_etudiant`) AS nb_etudiant
        FROM `etudiant`
        GROUP BY `ref_parcours`
        HAVING nb_etudiant <> 0;
```

Interrogation de base de données avec phpMyAdmin(18)

❑TRI (ORDER BY)

La commande ORDER BY permet de trier les lignes dans un résultat d'une requête SQL. Il est possible de trier les données sur une ou plusieurs colonnes, par ordre ascendant ou descendant.

- Syntaxe :

```
SELECT colonne1, colonne2
        FROM table
        ORDER BY colonne1 ordre

        Ordre : ASC ou DESC
```

- Exemple :

```
SELECT `nom`, `adresse`, `date_save`
        FROM `etudiant`
        ORDER BY `nom` ASC;
```

Interrogation de base de données avec phpMyAdmin(19)

❑ Limiter les nombres de résultat (LIMIT)

La clause LIMIT nous permet de spécifier le nombre maximum de résultats que l'ont souhaite obtenir. Elle est souvent associée à un OFFSET, c'est-à-dire effectuer un décalage sur le jeu de résultat.

- Syntaxe

```
SELECT colonne1, colonne2
        FROM table
        LIMIT nombre_ligne [OFFSET decalage] ou LIMIT decalage, nombre_ligne ou LIMIT nombre_ligne ;
```

- Exemple:

```
SELECT * FROM `etudiant`
        ORDER BY `nom` ASC
        LIMIT 2 OFFSET 1 ou LIMIT 1, 2 ;

SELECT * FROM `etudiant`
        WHERE `login` LIKE BINARY '%Tes%'
        ORDER BY `nom` ASC
        LIMIT 2 OFFSET 1
```

Interrogation de base de données avec phpMyAdmin(20)

❑ Liste de choix sous SQL (CASE … WHEN)

La commande « CASE … WHEN … » permet d'utiliser des conditions de type « si / sinon » similaire à un langage de programmation pour retourner un résultat disponible entre plusieurs possibilités. Le CASE peut être utilisé dans n'importe quelle instruction ou clause, telle que SELECT, UPDATE, DELETE, WHERE, ORDER BY ou HAVING.

○ Syntaxe

```
CASE a
        WHEN 1 THEN 'un'
        WHEN 2 THEN 'deux'
        WHEN 3 THEN 'trois'
        ELSE 'autre'
END
```

Ou

```
CASE
        WHEN a=b THEN 'A égal à B'
        WHEN a>b THEN 'A supérieur à B'
        ELSE 'A inférieur à B'
END
```

○ Exemple

```
SELECT `nom`, `adresse` ,
        CASE
                WHEN `ref_parcours` = 1 THEN 'Informatique 2023'
                WHEN `ref_parcours` = 2 THEN 'Gestion 2023'
                ELSE 'Parcours inconue'
        END AS nom_parc_annee
FROM `etudiant` ORDER BY `nom` ASC
```

Interrogation de base de données avec phpMyAdmin(21)

❑ La jointure

Les jointures en SQL permettent d'associer plusieurs tables dans une même requête. Il faut pour cela que les tables concernées par la jointure aient au moins chacune une colonne contenant un même type d'information.

- Jointure interne

  INNER JOIN : jointure interne pour retourner les enregistrements quand la condition est vrai dans les 2 tables.

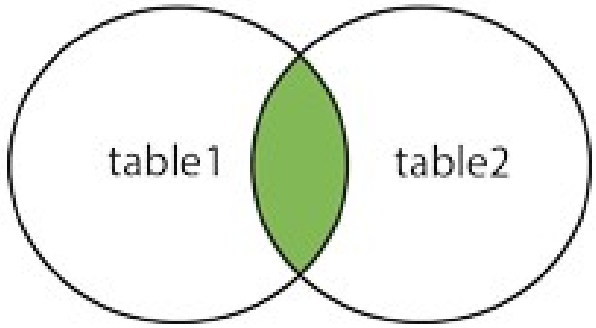
○ Syntaxe

```
SELECT *
FROM nom_table1
        INNER JOIN nom_table2
                ON colonne_table1 = colonne_table2
        [WHERE ...] [ORDER BY ...]
        [LIMIT ...];
```

Ou nom_table1, nom_table2

○ Exemple

```
SELECT et.nom, et.adresse, par.nom FROM `etudiant` as et
        INNER JOIN `parcours` as par
  ON et.`ref_parcours` = par.id_parcours
  ORDER BY et.nom ASC
```



Interrogation de base de données avec phpMyAdmin(22)

- Jointure externe

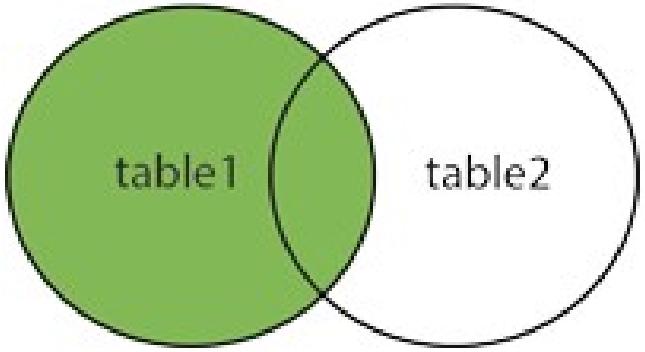
LEFT JOIN (ou LEFT OUTER JOIN) : jointure externe pour retourner tous les enregistrements de la table de gauche (LEFT = gauche) même si la condition n'est pas vérifié dans l'autre table.

- Syntaxe

```
SELECT *
FROM nom_table1
        LEFT JOIN nom_table2
                ON colonne_table1 = colonne_table2
        [WHERE ...] [ORDER BY ...]
        [LIMIT ...];
```

- Exemple

```
SELECT et.nom, et.adresse, par.nom FROM `parcours` as par
        LEFT JOIN  `etudiant` as et
   ON et.`ref_parcours` = par.id_parcours
   ORDER BY et.nom ASC
```



Interrogation de base de données avec phpMyAdmin(23)

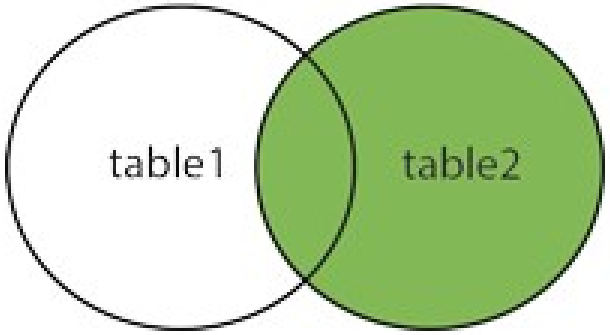
RIGHT JOIN (ou RIGHT OUTER JOIN) : jointure externe pour retourner tous les enregistrements de la table de droite (RIGHT = droite) même si la condition n'est pas vérifié dans l'autre table.

- Syntaxe

```
FROM nom_table1
    RIGHT JOIN nom_table2
            ON colonne_table1 = colonne_table2
    [WHERE ...] [ORDER BY ...]
    [LIMIT ...];
```

- Exemple

```
SELECT et.nom, et.adresse, par.nom FROM `etudiant` as et
    RIGHT JOIN `parcours` as par
  ON et.`ref_parcours` = par.id_parcours
  ORDER BY et.nom ASC
```



Interrogation de base de données avec phpMyAdmin(24)

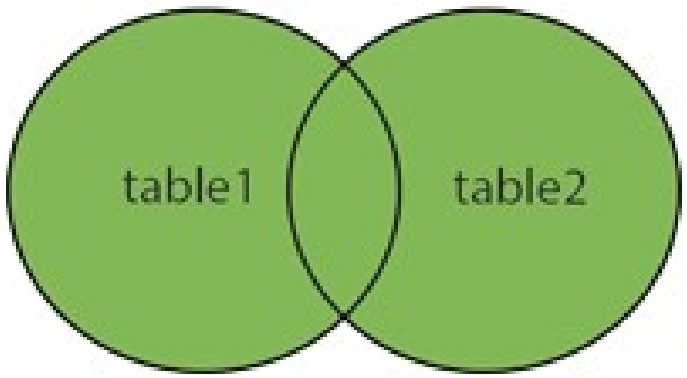
FULL JOIN : la commande FULL JOIN (ou FULL OUTER JOIN) permet de faire une jointure entre 2 tables. L'utilisation de cette commande permet de combiner les résultats des 2 tables, les associer entre eux grâce à une condition et remplir avec des valeurs NULL si la condition n'est pas respectée.

- Syntaxe

```
FROM nom_table1
        FULL JOIN nom_table2
                ON colonne_table1 = colonne_table2
        [WHERE ...] [ORDER BY ...]
        [LIMIT ...];
```

- Exemple

```
SELECT et.nom, et.adresse, par.nom FROM `etudiant` as et
        FULL JOIN `parcours` as par
    ON et.`ref_parcours` = par.id_parcours
    ORDER BY et.nom ASC
```



Interrogation de base de données avec phpMyAdmin(25)

- NATURAL JOIN : jointure naturelle entre 2 tables s'il y a au moins une colonne qui porte le même nom entre les 2 tables SQL.

  - Syntaxe

```
SELECT * FROM table1
         NATURAL JOIN table2;
```

Equivalent à

```
SELECT * FROM table1
         INNER JOIN table2
         ON table1.B = table2.B;
```

- Jointure avec USING

  Lorsque les colonnes qui servent à joindre les deux tables ont le même nom, nous pouvons utiliser la clause USING au lieu de la clause ON.

  - Syntaxe

```
SELECT *
         FROM table1
                  [INNER | LEFT | RIGHT] JOIN table2
                           USING (colonneJ);
```

Interrogation de base de données avec phpMyAdmin(26)

❑ Sous requête

Une sous requête (aussi appelé « requête imbriquée » ou « requête en cascade ») consiste à exécuter une requête à l'intérieur d'une autre requête.

○ Syntaxe

✓ Il y a plusieurs façons d'utiliser les sous-requêtes.

- Requête imbriquée qui retourne un seul résultat

```
SELECT *
        FROM `table`
        WHERE `nom_colonne` =
                ( SELECT `valeur` FROM `table2` LIMIT 1 )
```

- Requête imbriquée qui retourne une colonne

```
SELECT *
        FROM `table`
        WHERE `nom_colonne` IN
                ( SELECT `colonne` FROM `table2` WHERE `cle_etrangere` = ? )
```

Avec ? : id de la clé étrangère